MASTERCARD

Interview Study Guide

Software Development Engineer II — Data & Analytics

Business Experimentation & Optimisation Team

Idhaya Bastine Kennedy

May 2026 | Dublin, Ireland

Covers: JD Concepts · Project Code · Architecture · A/B Testing · STAR Stories · Interview Cheat Sheets · Gap Framing

Section	Topic	Priority
1	JD → Project Evidence Map	Read first
2	Full Architecture Deep Dive	Read second
3.1	PySpark & Distributed Computing	HIGH
3.2	ETL Pipeline & Parquet	HIGH
3.3	A/B Testing & Experimentation	CRITICAL ★
3.4	Broadcast Joins & Optimisation	HIGH
3.5	Data Warehousing Patterns	HIGH
3.6	REST API Design (FastAPI)	MEDIUM
3.7	Orchestration with Dagster	MEDIUM
4.1	Databricks & Delta Lake (GAP)	HIGH

4.2	Kafka & Streaming (GAP)	MEDIUM
4.3	Idempotency (GAP — Bar Raiser)	HIGH
4.4	Dead Letter Queues (GAP)	MEDIUM
5	All 6 STAR Stories	Practise aloud
6	Interview Cheat Sheets (3 rounds)	Day-before read
7	Quick Reference Glossary	Quick scan

Section 1: JD → Project Evidence Map

Every claim you make in the interview must be backed by concrete evidence. This table maps each JD requirement to your match strength and the exact evidence from your project and TRIAS experience.

JD Requirement	Match	Your Evidence
2+ yrs full-stack, production-grade apps	STRONG	3 yrs TRIAS · 15+ features end-to-end · 100k+ daily users
Microservices & RESTful APIs	STRONG	api/main.py FastAPI · /api/v1/ versioned routes · Pydantic validation · TRIAS mic
Relational DBs & distributed data stores	STRONG	PostgreSQL (prod) · SQLite (demo) · SQLAlchemy ORM · Elasticsearch at TRIA
Cloud — AWS and/or Azure	STRONG	AWS (Lambda, S3, EC2, RDS) · Azure extended at TRIAS · Docker on AWS for
Docker & Kubernetes	STRONG	Dockerfile + docker-compose.yml in project · K8s CI/CD at TRIAS · 25% faster d
Databricks & PySpark — pipelines & workflows	PARTIAL ★	PySpark 3.5 pipeline · partitioned Parquet · Spark SQL · broadcast join · Same c
A/B Testing / Experimentation platform	PARTIAL ★	ab_engine.py · Welch t-test · Cohen's d · 95% CI · pairwise region experiments –
GenAI / LLM concepts (a plus)	STRONG	OpenAI API · RAG · MCP Protocols · LLM agents at TRIAS · 50% processing tim
Java (Mastercard core language)	PARTIAL	Listed as 'familiar' · honest framing: Python/Node strong · fast-learner track recor
Cross-functional: Product, QA, Data Science	STRONG	Verbatim in resume · global teams · 2 international deployments
Design & code reviews, TDD, security	STRONG	30% bug reduction in 6 months · design review process · automated linting in CI
Globally distributed teams, Agile	STRONG	India · Papua New Guinea · Dublin · async-first patterns · sprint delivery

★ Two gaps that need the most preparation

Databricks: You have strong PySpark. What you need to say: 'My PySpark code in this project runs unchanged on Databricks — I'd swap SparkSession local[*] for a cluster config and Parquet writes for Delta table writes. I'm actively deepening platform knowledge: notebooks, Delta Lake, Unity Catalog, Workflows.'

A/B Testing: You BUILT an experimentation engine in this project. ab_engine.py IS the BE&O; team's core work. You need to speak fluently about Welch t-test, p-value, Cohen's d, confidence intervals. Section 3.3 covers all of this in depth.

Section 2: Full Architecture Deep Dive

This project is a production-pattern data engineering platform. Every layer mirrors something that exists at Mastercard scale. Understanding WHY each layer exists is as important as knowing what it does.

Full Pipeline Architecture

2.1 What Each Layer Maps to in Production (Mastercard Scale)

This Project Layer	Production Equivalent	Why It Matters for the Interview
pipeline/ingest.py	Databricks Ingest Job / Kafka Consumer	Shows you understand the raw→staged pattern and schema enforceme

data/staging/ Parquet	Delta Lake Bronze table	Bronze = raw, schema-on-read. Your Parquet is the equivalent.
pipeline/transform.py	Databricks Silver table job	Silver = cleaned, enriched. Broadcast join, dedup, derived columns.
data/processed/ partitioned Parquet	Delta Lake Silver table (partitioned)	Partition pruning = same performance concept as predicate pushdown
pipeline/load.py + Spark SQL	Databricks Gold table job	Gold = aggregated, business-ready. Spark SQL GROUP BY before colle
experiments/ab_engine.py	Mastercard BE&O Experimentation Engine	THIS IS THE ROLE. Welch t-test, effect size, CI — the exact statistical c
orchestration/etl_job.py Dagster	Databricks Workflows / Airflow	Typed op outputs = Databricks task dependencies. Same concept, differ
api/main.py FastAPI	Internal Analytics API / Microservice	Shows full-stack: pipeline output served via REST, versioned, self-docu
docker-compose.yml	Kubernetes + Helm charts in prod	Shows infra thinking: reproducible, environment-consistent deployments

2.2 Medallion Architecture — Bronze / Silver / Gold

What is Medallion Architecture?

A data lake design pattern (originated at Databricks) that organises data into three quality layers. Your project implements all three:

Bronze (Raw): data/staging/ — raw Parquet with schema enforcement. No transformations yet. Source of truth for reprocessing.

Silver (Cleaned): data/processed/ — deduplicated, enriched, null-filled, string-normalised, partitioned by region. Safe to query.

Gold (Aggregated): regional_cost_statistics table — GROUP BY aggregates ready for dashboards and APIs. No heavy computation at query time.

Say this when asked about data architecture:

"The pipeline follows a Medallion pattern — raw ingestion to a staging zone, then a cleaned Silver layer partitioned by region for predicate pushdown, and finally Gold aggregates computed in Spark SQL before writing to PostgreSQL. In production I'd replace the Parquet layers with Delta tables for ACID guarantees and time-travel."

Section 3: Core Concepts — With Your Code

3.1 PySpark & Distributed Computing

What is Apache Spark? (explain it like this)

The problem it solves: A single machine can't process terabytes of data fast enough. Spark distributes the work across a cluster of machines.

Driver: Your Python script. It plans what to do — it never touches the data directly.

Executors: Worker machines that actually process the data partitions in parallel.

DAG (Directed Acyclic Graph): Spark builds a graph of all your transformations before running any of them. This lets it optimise the plan. It is LAZY — nothing runs until you call an action like .count() or .write().

Catalyst Optimiser: Spark's query planner. It rewrites your DataFrame operations into the most efficient execution plan — like a SQL query optimiser but for DataFrames.

local[*]: In your project, Spark runs on your laptop using all CPU cores. On Databricks, you'd swap this for a managed cluster — the code is identical.

pipeline/ingest.py — SparkSession & Schema

```
def get_spark() -> SparkSession:
    return (
        SparkSession.builder
        .appName("CostOfLivingPipeline")
        .master("local[*]")             # uses all CPU cores locally
                                         # → on Databricks: .master("cluster-url")
        .config("spark.sql.adaptive.enabled", "true")   # AQE: auto-optimises at runtime
        .config("spark.sql.shuffle.partitions", "8")    # small dataset → 8 not 200
        .getOrCreate()
    )

RAW_SCHEMA = StructType([
    StructField("case_id",              IntegerType(), True),  # explicit type
    StructField("housing_cost",         FloatType(),   True),  # not inferred
    StructField("total_cost",           FloatType(),   True),  # prevents silent coercion
    StructField("median_family_income", FloatType(),   True),
    # ... all 14 columns defined
])
# WHY: Without a schema, Spark reads the whole file to infer types (expensive)
# and may silently turn numeric columns into strings.
```

In This Project: pandas bridge pattern

The source file is an XLS (Excel-format CSV). There is no native Spark XLS connector, so pandas reads it first (fast for a 3,000-row file), then hands the DataFrame to Spark with .createDataFrame(raw_pd, schema=RAW_SCHEMA). This is a standard pattern for small-source → large-engine pipelines.

When asked 'Tell me about your PySpark experience':

"In this project I built a full PySpark pipeline — ingest, transform, load. I used schema enforcement with StructType to prevent silent type coercion, broadcast joins to avoid shuffles on the state lookup, and partitioned Parquet output so downstream queries use predicate pushdown. The same code runs unchanged on Databricks — I'd just swap local[*] for a cluster config and Parquet writes for Delta writes."

3.2 ETL Pipeline Design & Parquet

ETL — Extract, Transform, Load

Extract: Read raw data from source systems (CSV, database, API, Kafka topic). Apply schema. Write to staging zone without transforming.

Transform: Clean (nulls, dedup), enrich (derived columns, lookups), validate (type checks, range checks), aggregate. Write to processed zone.

Load: Write the final, analytics-ready data to the target system (data warehouse, PostgreSQL, dashboard). Aggregations happen before writing.

Why Parquet? (not CSV)

Columnar storage: Data is stored column-by-column, not row-by-row. If your query only needs housing_cost and region, Parquet reads only those columns — skipping all others. CSV reads every column even if you only need two.

Compression: Columnar layout compresses much better because similar values are stored together. Parquet files are typically 5-10× smaller than equivalent CSV.

Predicate pushdown: When you partition by region and query WHERE region='West', Parquet skips all non-West partition folders entirely — never reads them.

Schema embedded: The data types are stored in the file. No guessing.

Feature	CSV	Parquet	Delta Lake
Storage	Row-based	Columnar	Columnar (Parquet + log)
Compression	None	High	High
Schema	Inferred	Embedded	Enforced + versioned
Partial reads	No	Yes	Yes
ACID	No	No	Yes
Time travel	No	No	Yes (versions)
Updates/Deletes	No	No	Yes (MERGE/DELETE)
Streaming	Batch only	Batch only	Batch + Streaming

Idempotency — critical for pipeline reliability

Definition: Running the same pipeline twice produces exactly the same result as running it once. No duplicate rows. No corrupted state.

How this project achieves it:

→ df.write.mode('overwrite').parquet(path) — replaces the Parquet folder entirely

→ to_sql(..., if_exists='replace') — drops and recreates the PostgreSQL table

→ Deduplication on county in transform layer — safe to re-run

Why it matters: Batch jobs WILL fail and be retried. If your pipeline isn't idempotent, a retry causes duplicate rows in your data warehouse. This breaks all downstream analytics. Mastercard runs billions of transactions — idempotency is non-negotiable.

When the Bar Raiser asks 'How do you ensure idempotency?':

"In this project, every write uses mode='overwrite' for Parquet and if_exists='replace' for PostgreSQL — so re-running the full pipeline is safe. For production I'd go further: partition-level overwrite on Delta tables (replaceWhere) so we only rewrite the affected date/region partition rather than the whole table. Plus checkpointing in Spark Structured Streaming so after a failure we resume from exactly where we stopped."

3.3 A/B Testing & Statistical Experimentation ★ CRITICAL

This is the core work of the BE&O; team. You BUILT this in ab_engine.py. You must be able to explain every statistical concept clearly and connect it to your code. Read this section multiple times.

What is an A/B Test? (start here)

An A/B test is an experiment where you split a population into two groups — a **Control group** (current state, e.g. Midwest counties) and a **Treatment group** (new state, e.g. Northeast counties) — and measure whether a metric differs significantly between them.

In your project: you compare cost-of-living metrics between US Census regions. Is the West's average total_cost statistically significantly higher than the Midwest's? Or could the difference be random noise?

In Mastercard's context: did adding Feature X to the app significantly change transaction volume? Did a new pricing model significantly affect customer retention?

The Null Hypothesis and p-value — explained plainly

Null Hypothesis (H■): 'There is NO real difference between the two groups. Any observed difference is just random chance.'

Alternative Hypothesis (H■): 'There IS a real difference.'

p-value: The probability of seeing a difference as large as the one you observed, IF the null hypothesis were true (i.e. if there were really no difference).

p = 0.03 means: If there truly were no difference, you'd see a gap this large only 3% of the time by chance. That's unlikely → we reject H■ → significant result.

p = 0.40 means: You'd see this gap 40% of the time by chance. → Not convincing → fail to reject H■ → NOT significant.

Alpha (α = 0.05): Your threshold. If p < 0.05, the result is 'statistically significant'. This is the industry standard (5% false positive rate acceptable).

Why Welch's t-test? (not Student's t-test)

Student's t-test assumption: Both groups have equal variance (spread). This is almost never true in real-world data.

Welch's t-test: Does NOT assume equal variance. It adjusts the degrees of freedom using the Welch-Satterthwaite equation. More robust. Always preferred when group sizes or variances differ.

In your data: Northeast has 103 counties, Midwest has 516. Variance in costs is different across regions. Welch's is the correct choice.

Code: stats.ttest_ind(control, treatment, equal_var=False) — equal_var=False is what makes it Welch's.

Cohen's d — Effect Size

p-value tells you IF there's a difference. Cohen's d tells you HOW BIG it is.

Formula: d = (treatment_mean – control_mean) / pooled_std

Interpretation: Small < 0.2 | Medium < 0.5 | Large ≥ 0.8

Example: p=0.0001 but d=0.05 means: yes it's significant, but the actual difference is tiny — 0.05 standard deviations. With a large enough sample, even trivial differences become statistically significant. Cohen's d keeps you honest.

In your results: Midwest vs West on total_cost: p=0.0000, d=1.239 → Large effect. This is a real, meaningful difference, not just a statistical artefact.

95% Confidence Interval — what it means

The 95% CI on the mean difference gives you a range: 'The true difference between these two groups is probably between CI_lower and CI_upper.'

If the CI does NOT cross zero: the difference is statistically significant (consistent with p < 0.05). One group is clearly higher/lower.

If the CI crosses zero: zero difference is plausible — not significant.

Example: CI = [2607, 4707] for Midwest vs Northeast total_cost → Northeast costs $2,607 to $4,707 more per year than Midwest. Both bounds positive → significant.

Welch-Satterthwaite formula used: Adjusts degrees of freedom for unequal variances before computing t_crit for the CI bounds.

Lift % — business language for the difference

Lift % = (treatment_mean – control_mean) / control_mean × 100

This converts the raw dollar difference into a percentage change from the control baseline.

Example: Midwest avg total_cost = $35,036. West = $39,669. Lift = ($39,669 – $35,036) / $35,036 × 100 = +13.2%. The West is 13.2% more expensive than the Midwest.

Why this matters for Mastercard: Lift % is the language product teams and executives understand. 'Our new feature increased transaction volume by 8.3%' — that's lift.

experiments/ab_engine.py — full statistical core

```
def run_experiment(engine, control_region, treatment_region,
                   metric="total_cost", alpha=0.05):

    # 1. Pull data from DB
    df = pd.read_sql("SELECT region, {metric} FROM cost_of_living_processed "
                     "WHERE region IN (:c, :t)", conn,
                     params={"c": control_region, "t": treatment_region})

    control   = df[df["region"]==control_region][metric].dropna().values
    treatment = df[df["region"]==treatment_region][metric].dropna().values

    # 2. Welch's t-test (equal_var=False = Welch's, not Student's)
    t_stat, p_value = stats.ttest_ind(control, treatment, equal_var=False)

    # 3. Cohen's d — effect size (how BIG is the difference?)
    pooled_std = np.sqrt((control.std()**2 + treatment.std()**2) / 2)
    cohens_d   = (treatment.mean() - control.mean()) / pooled_std

    # 4. 95% Confidence Interval on mean difference (Welch-Satterthwaite)
    mean_diff = treatment.mean() - control.mean()
    se_diff   = np.sqrt(control.std()**2/len(control) +
                        treatment.std()**2/len(treatment))
    # Welch degrees of freedom
    df_welch = (se_diff**2)**2 / (
        (control.std()**2/len(control))**2/(len(control)-1) +
        (treatment.std()**2/len(treatment))**2/(len(treatment)-1))
    t_crit   = stats.t.ppf(1 - alpha/2, df=df_welch)  # 1.96 approx for large n
    ci_lower = mean_diff - t_crit * se_diff
    ci_upper = mean_diff + t_crit * se_diff
```

```
    # 5. Lift % — % change from control to treatment
    lift_pct = mean_diff / control.mean() * 100

    # 6. Significance & Winner
    significant = bool(p_value < alpha)          # p < 0.05 → significant
    if significant:
        # For costs: lower is better. For affordability_ratio: higher is better.
        higher_is_better = (metric == "affordability_ratio")
        treatment_wins = (treatment.mean() > control.mean()) if higher_is_better                    else (treatm
        winner = treatment_region if treatment_wins else control_region
    else:
        winner = "no_winner"
```

How to talk about A/B testing at Mastercard (this is the money answer):

"In my project I built an experimentation engine that runs pairwise Welch's t-tests across all US Census regions on cost-of-living metrics. I chose Welch's over Student's because the groups have unequal sizes and variance — Midwest has 516 counties, Northeast only 103. Each result includes p-value, Cohen's d effect size, 95% confidence interval, and lift percentage. That gives you both statistical significance AND practical significance — a p-value alone can be misleading with large samples."

"The architecture mirrors what the BE&O; team does: you have a metric store (PostgreSQL), an experiment runner (ab_engine), and an API to query results. I'd extend this with multi-armed bandit algorithms and sequential testing for early stopping."

3.4 Broadcast Join & Query Optimisation

Why shuffles are expensive in distributed computing

In a distributed cluster, a JOIN normally requires a shuffle — every machine must send its relevant rows to the machine that has the matching rows from the other table. Network I/O is the bottleneck. A shuffle on a 100GB table can take minutes.

Broadcast join solution: If one table is small (e.g. a 50-state lookup table), send that ENTIRE small table to every worker. Each worker can then do its join locally without any network I/O. Spark does this automatically when the smaller table fits in memory, but F.broadcast() forces it explicitly.

pipeline/transform.py — Broadcast Join

```
# Build a small lookup: ('AL', 'Alabama'), ('CA', 'California'), ...
abbrev_rows = [(k, v) for k, v in STATE_ABBREV.items()]   # 50 rows — tiny
abbrev_df   = spark.createDataFrame(abbrev_rows, ["abbrev", "full_name"])

df = (
    df.join(
        F.broadcast(abbrev_df),          # << force broadcast — send to all workers
        df["state"] == abbrev_df["abbrev"],
        "left"                           # keep counties even if abbrev not found
    )
    .withColumn("state", F.coalesce(F.col("full_name"), F.col("state")))
    .drop("abbrev", "full_name")
)
# Result: 'AL' → 'Alabama', 'CA' → 'California', etc.
```

Predicate Pushdown with Partitioned Parquet

When you write: df.write.partitionBy('region').parquet('data/processed/'), Spark creates separate folders: region=West/, region=Midwest/, region=Northeast/, etc.

When a query later reads: spark.read.parquet('data/processed/').filter(region='West'), Spark's Catalyst optimiser pushes the filter DOWN to the file reader — it only opens the region=West/ folder, never touching the others.

For 5 regions, this is a 5× speedup on region-filtered queries. At Mastercard scale with hundreds of partitions, predicate pushdown is the difference between a 2-second query and a 2-hour one.

When asked 'How do you optimise Spark jobs?':

"Three main levers: 1) Broadcast joins for small lookup tables to eliminate shuffles, 2) Partitioning output by high-cardinality filter columns (like region or date) for predicate pushdown on reads, 3) Adaptive Query Execution (AQE) which I enable in the SparkSession config — it reoptimises the plan at runtime based on actual data statistics rather than estimates."

3.5 Data Warehousing Patterns

The three-zone architecture used in this project

Staging Zone (data/staging/): Raw Parquet from ingest. Schema-enforced but otherwise untransformed. Purpose: reproducible starting point for re-processing.

Processed Zone (data/processed/): Cleaned, enriched, partitioned. Safe for analysts to query. Contains derived columns (affordability_ratio, region).

Analytical Layer (PostgreSQL tables): Pre-aggregated for fast API responses. regional_cost_statistics has only 5 rows — /api/v1/regions responds in milliseconds.

pipeline/load.py — Spark SQL aggregation stays on cluster

```
# WRONG approach (don't do this):
# county_pd = df.toPandas()            # pulls ALL 1,600 rows to driver
# regional = county_pd.groupby(...)    # runs on single machine in Python

# CORRECT approach (what this project does):
# GROUP BY runs DISTRIBUTED on Spark executors → only 5 summary rows collected
regional_df = spark.sql("""
    SELECT
        region,
        ROUND(AVG(total_cost),           2) AS avg_total_cost,
        ROUND(AVG(housing_cost),         2) AS avg_housing_cost,
        ROUND(AVG(median_family_income), 2) AS avg_median_income,
        ROUND(AVG(affordability_ratio),  3) AS avg_affordability_ratio,
        COUNT(*)                            AS county_count
    FROM cost_of_living                -- Spark temp view, not PostgreSQL
    WHERE region != 'Other'
    GROUP BY region
    ORDER BY avg_total_cost DESC
""")
regional_pd = regional_df.toPandas()  # only 5 rows cross the wire
regional_pd.to_sql("regional_cost_statistics", engine, if_exists="replace")
# At Mastercard scale: GROUP BY on Spark over billions of rows,
# then write a tiny summary table to the serving layer.
```

3.6 REST API Design with FastAPI

Why FastAPI? (not Flask or Django)

Pydantic validation: Request bodies are validated automatically. If you send the wrong type, FastAPI returns a 422 before your code runs.

Auto OpenAPI docs: /docs gives you a live Swagger UI. No extra code needed. Every endpoint, parameter, and model is documented automatically.

Type hints = documentation: Python type hints + Pydantic models = self-documenting API that's easy for other teams to consume.

SQL injection prevention: SQLAlchemy text() with named params (e.g. WHERE region = :metric) prevents injection — user input never concatenated into SQL.

api/main.py — key patterns

```
# Pydantic model — validates POST /experiments/run body
class ExperimentRequest(BaseModel):
    control_region:   str
    treatment_region: str
    metric: str   = "total_cost"   # default value
    alpha:  float = 0.05           # type + default = auto-validated

# Parameterised query — SQL injection safe
rows = conn.execute(
    text("""SELECT * FROM experiment_results
            WHERE metric = :metric
            AND (:sig_only = false OR significant = true)"""),
    {"metric": metric, "sig_only": significant_only}   # params, not f-string
)

# Connection pool — pool_pre_ping=True reconnects if DB drops
engine = create_engine(DATABASE_URL, pool_pre_ping=True)

# Versioned route — /api/v1/ allows future breaking changes in /api/v2/
@app.get("/api/v1/regions", tags=["analytics"])
def get_regional_stats(): ...
```

When asked about microservice / API design patterns you use:

"A few patterns I apply: versioned routes (/api/v1/) so breaking changes go to v2 without breaking existing clients; Pydantic models at the boundary for input validation so bad requests fail fast with clear error messages; parameterised queries everywhere to prevent SQL injection; connection pool with pre-ping for resilience against transient DB disconnects. The /health endpoint is there for orchestrators like Kubernetes or load balancers to probe."

3.7 Orchestration with Dagster

Why orchestration? (not just a cron job)

Without orchestration: You run scripts manually or via cron. If step 2 fails, you don't know. You re-run the whole thing. You have no record of what ran when or what the data looked like.

With Dagster: Every run is logged. Failed ops are retried automatically. You see the lineage — which data assets depend on which. Typed outputs mean step 3 can't start if step 2 returned the wrong type.

How it compares to Databricks Workflows: Same concept. Databricks Workflows = Tasks with dependencies, triggered on schedule or event. Dagster @op = Databricks Task. @job = Databricks Workflow.

orchestration/etl_job.py — Dagster typed ops

```
@op(out=Out(int))            # declares: this op outputs an integer
def ingest_op() -> int:
    count = ingest(SOURCE_PATH, STAGING_PATH)
    return count             # Dagster validates this is actually an int

@op(ins={"ingested": In(int)},   # declares: takes the int from ingest_op
    out=Out(int))
def transform_op(ingested: int) -> int:
    count = transform(STAGING_PATH, PROCESSED_PATH)
    return count             # Dagster wires: ingest_op.output → transform_op.input

@job
def etl_pipeline():
    experiment_op(           # dependency chain: ingest → transform → load → experiments
        load_op(
            transform_op(
                ingest_op()  # innermost runs first
    )))

# Run it:
# dagster job execute -f orchestration/etl_job.py -j etl_pipeline
```

Section 4: Beyond the Project — Gap Concepts

These are concepts the JD requires that your project partially or doesn't fully cover. You must be able to speak to them in the interview. Each section gives you the concept, how it extends your project, and exactly what to say.

4.1 Databricks & Delta Lake ★ HIGH PRIORITY GAP

The gap: you know PySpark but not the full Databricks platform

Your PySpark code is solid. What the interviewer wants to hear is that you understand what Databricks ADDS on top of raw Spark, and that your code is deployment-ready.

What Databricks adds over raw PySpark

Managed clusters: No JVM setup. One config change deploys your SparkSession to a cloud cluster (AWS/Azure). Auto-scaling. Your code: change .master('local[*]') to .master('databricks') or use the Databricks Connect library.

Notebooks: Interactive development with real data. DataFrame results render as tables. Like Jupyter but cluster-backed.

Unity Catalog: Centrally governed data catalog. Tables, schemas, access control. Your PostgreSQL tables would become Unity Catalog managed tables.

Databricks Workflows: Dagster @job equivalent. Schedule ETL jobs, set dependencies, receive alerts. Your etl_job.py maps directly.

MLflow: Built-in experiment tracking. Log parameters, metrics, models. Your ab_engine.py experiment results would be MLflow runs.

Delta Lake — what it is and why it matters

Delta Lake is Parquet + a transaction log. Every write is recorded in _delta_log/. This gives you:

ACID transactions: Concurrent reads and writes don't corrupt data. Critical when multiple pipelines write to the same table.

Time travel: SELECT * FROM my_table VERSION AS OF 5 — read the table as it was 5 versions ago. Audit, rollback, reproduce historical results.

Schema enforcement: Delta rejects writes that don't match the table schema. No silent data corruption.

MERGE / UPSERT: MERGE INTO cost_of_living USING new_data ON id=id WHEN MATCHED UPDATE ... WHEN NOT MATCHED INSERT ... — atomic upserts without delete+reinsert.

Streaming + Batch same table: Spark Structured Streaming can write to a Delta table while a batch job reads from it. Concurrent, safe.

How to migrate this project to Databricks + Delta Lake

Ingest layer: Replace spark.read.csv() with Databricks Auto Loader (cloud_files) — monitors S3/ADLS for new files and processes incrementally.

Staging write: df.write.mode('overwrite').parquet(staging_path) → df.write.format('delta').mode('append').saveAsTable('bronze.cost_of_living')

Transform write: partitionBy('region').parquet(...) → .write.format('delta').partitionBy('region').saveAsTable('silver.cost_of_living')

Load layer: Spark SQL GROUP BY same → write to delta table 'gold.regional_stats'

Orchestration: Dagster @job → Databricks Workflow with 4 tasks

Exact sentence to say when Databricks comes up:

"My PySpark code in this project runs unchanged on Databricks — the SparkSession, DataFrame operations, and Spark SQL are all standard PySpark. To move to Databricks production I'd make three changes: swap local[*] for a Databricks cluster config, replace Parquet writes with Delta table writes for ACID guarantees and time-travel, and replace Dagster with Databricks Workflows for managed scheduling and alerting. I'm actively working through the Databricks Community Edition to deepen my platform knowledge — I'm focused on Unity Catalog, Auto Loader, and Delta MERGE patterns."

4.2 Kafka & Streaming — Batch vs Stream

Batch vs Streaming — when to use which

Batch processing (what this project does): Process all the data at once on a schedule — hourly, daily, weekly. Simple, easier to debug, good for large historical datasets. Your ETL pipeline: runs once, processes ~3,000 rows.

Stream processing: Process each event as it arrives — milliseconds latency. Required when business decisions must happen in real-time (fraud detection, live dashboards, personalisation).

Kafka: A distributed message broker. Producers write events to topics (e.g. 'cost-updates'). Consumers read and process them. Events are durable and replayable — you can re-process from any offset.

How ingest.py would change for streaming

Replace: pd.read_csv(source_path) → spark.readStream.format('kafka')...

The transform logic stays the same — PySpark DataFrames work for both batch and stream.

Replace: df.write.parquet(staging) → df.writeStream.format('delta').start()

Trigger: either micro-batch (process every 30 seconds) or continuous.

This is the 'Planned Extension — Streaming ingestion' mentioned in the README.

When asked about streaming experience:

"In this project the pipeline is batch — scheduled ingest of the full dataset. I designed it so the PySpark transform logic is streaming-compatible: the same DataFrame operations work on readStream. The planned extension is to replace the batch ingest with a Spark Structured Streaming consumer on a Kafka topic, processing new county records as they arrive. The architecture stays the same — just swap the source and sink APIs."

4.3 Idempotency — Bar Raiser Deep Dive

Three levels of idempotency to know

Level 1 — Full table replace (this project): if_exists='replace' / mode='overwrite'. Simple. Safe. But rewrites everything even if only 1% changed. Fine for small tables.

Level 2 — Partition-level overwrite: df.write.mode('overwrite').option('replaceWhere', 'region=West AND date=2024-01-01').parquet(...). Only the West/2024-01-01 partition is rewritten. Faster, more targeted.

Level 3 — MERGE / UPSERT (Delta Lake): MERGE INTO target USING source ON target.id = source.id WHEN MATCHED UPDATE ... WHEN NOT MATCHED INSERT ... Row-level precision. Only changed rows are written. True production pattern.

Bar Raiser: 'How would you design this pipeline to handle late-arriving data?'

"Late-arriving data — say a county's cost data arrives 2 days after the batch ran — is handled at Level 3 with Delta MERGE. The pipeline uses the case_id as the merge key: if the record already exists, UPDATE it; if it's new, INSERT it. The partition key is date or region so the MERGE only scans the relevant partition. For the experiment layer, I'd re-run only the experiments that involve the affected region rather than all 30 pairwise comparisons."

4.4 Dead Letter Queues & Fault Tolerance

What is a Dead Letter Queue (DLQ)?

A DLQ is a holding area for messages or records that failed processing. Instead of crashing the whole pipeline when one bad record arrives, you route the bad record to the DLQ, log it, and continue processing the rest.

Example: 3,000 county records arrive. One has NULL in total_cost and causes a division-by-zero in affordability_ratio. Without DLQ: pipeline crashes. With DLQ: that one record goes to an error_records table, the other 2,999 process normally.

In Kafka: A DLQ is literally a separate Kafka topic (e.g. 'cost-updates-dlq'). Failed messages are published there for investigation and replay.

How this project already implements lightweight fault tolerance

In ab_engine.py, run_all_experiments() wraps each experiment in try/except:

for control, treatment in pairs: try: results.append(run_experiment(...)) except ValueError as e: print(f'Skipped {control} vs {treatment}: {e}')

This is the DLQ pattern: one failing experiment (e.g. insufficient data for a pair) doesn't crash the whole run. The error is logged and processing continues.

When the Bar Raiser asks about fault tolerance in pipelines:

"I use a few patterns: For record-level failures, I route bad records to an error table (DLQ equivalent) rather than failing the whole batch — the ab_engine already does this with try/except per experiment. For job-level failures, Dagster handles retries with configurable retry policies on each op. For data corruption, Delta Lake's transaction log means I can roll back to a known-good version. For the API layer, pool_pre_ping handles transient DB disconnects, and the /health endpoint lets orchestrators detect and restart unhealthy pods."

4.5 Microservices & Production API Patterns

Patterns already in this project (point these out in interviews)

Health check endpoint: GET /health → {'status': 'ok'}. Kubernetes liveness probes, load balancers, and Dagster sensors all use this to know if the service is up.

Versioned API: /api/v1/ prefix. Breaking changes go to /api/v2/ without breaking existing clients.

Connection pool: create_engine(url, pool_pre_ping=True). Pre-ping tests the connection before each query — automatically reconnects if the DB dropped the idle connection.

Input validation at boundary: Pydantic ExperimentRequest validates control_region, treatment_region, metric before any business logic runs.

Parameterised queries: text('WHERE metric = :metric') + params dict. Never f-string user input into SQL — prevents SQL injection.

Section 5: All 6 STAR Stories

Practise each story out loud. Target: 90 seconds per story. Numbers are your anchors — never forget the metric. End every story on the result, not the action.

Story 1 — Performance Optimisation

Use for: Technical challenge · Problem-solving · Data-driven decision making · Driving impact

S — Situation

The main clinical dashboard at TRIAS had page load times of around 5 seconds. For clinicians in time-critical healthcare workflows, this was a serious usability problem affecting 100,000+ daily active users across multiple international deployments.

T — Task

Investigate the root cause and fix the performance issue without causing regressions on a live production platform — with no scheduled downtime window available.

A — Action

I profiled the full request chain and identified two root causes: an N+1 query problem in the microservice layer where each record triggered a separate database call, and a missing cache on a high-frequency API endpoint. I refactored the query logic to use batched joins, implemented Redis caching on the most-called routes, and set up performance benchmarking to validate the fix before deploying to production.

R — Result

Page load time dropped from 5 seconds to under 1 second — an 80% improvement. Zero regressions across the release. The fix was deployed with no downtime and held under load across both the Indian and Papua New Guinea deployments.

Story 2 — CI/CD & Deployment Pipeline

Use for: Engineering initiative · Reducing friction · DevOps ownership · Quantified impact

S — Situation

Deployment at TRIAS was a slow, largely manual process. Releases were infrequent and high-risk, particularly given live international healthcare clients with zero tolerance for production downtime.

T — Task

Design and implement a CI/CD pipeline that would reduce deployment cycle time, increase release confidence, and enable the team to ship faster without increasing risk.

A — Action

I designed a CI/CD pipeline using Jenkins, Docker, and Kubernetes on AWS — automating build, test, and deployment stages. I introduced containerisation so environments were consistent from dev to prod, and added automated rollback triggers so any failed deployment would self-recover. I also extended the same pipeline architecture to Azure for our cross-platform client requirements.

R — Result

Deployment cycle time dropped by 25%. We went from infrequent, high-stress releases to regular, reliable deployments — with zero production downtime across all subsequent releases. The team's confidence in shipping increased significantly.

Story 3 — GenAI Integration into Production

Use for: Innovation · Technical direction · Emerging technology · Measurable impact

S — Situation

TRIAS had a significant amount of manual data processing work — clinical staff were spending considerable time on tasks that were essentially pattern-matching against structured records. There was no AI capability in the platform.

T — Task

Evaluate whether Generative AI could automate or accelerate these workflows, and if so, design and integrate a production-grade solution — not a prototype.

A — Action

I researched LLM agent patterns and RAG architectures, then built a proof of concept using the OpenAI API and Model Context Protocol (MCP) integrated with our existing data layer. After validating the results with the clinical team, I productionised it — adding error handling, logging, and graceful degradation for cases where the model was uncertain. I also ran before/after measurement to quantify the actual impact.

R — Result

Data processing time for the targeted workflows dropped by 50%. The integration became the foundation for the platform's AI strategy, and I established the technical direction and code standards for all subsequent GenAI work at TRIAS.

Story 4 — Global Cross-functional Delivery

Use for: Collaboration · Stakeholder management · Delivering under complexity · Cross-team alignment

S — Situation

TRIAS was preparing an international release for a healthcare client in Papua New Guinea — a completely different regulatory environment, different infrastructure constraints, and a time-zone gap that made synchronous collaboration difficult.

T — Task

Deliver 15+ production features across a complex, multi-tenant healthcare platform on time, coordinating across Engineering, Product, QA, and Data Science teams spread across multiple time zones.

A — Action

I owned the technical delivery end-to-end — from architecture design reviews through to production deployment. I set up async communication patterns so dependencies didn't block progress across time zones, ran regular syncs with QA to align on acceptance criteria early, and worked closely with the Product team to manage scope creep without sacrificing quality. I also built onboarding documentation and ran technical presentations to bring newer team members up to speed on the complex modules.

R — Result

Delivered 2 international releases — India and Papua New Guinea — with zero critical defects. Deployment was on schedule. The async-first patterns I introduced reduced cross-team blockers and were adopted as the default working model for subsequent international projects.

Story 5 — Code Quality & Engineering Culture

Use for: Raising the bar · Ownership · Long-term thinking · Engineering best practices

S — Situation

Post-release bugs were a recurring problem at TRIAS. The team was shipping code that worked in dev but caused issues in production — partly due to inconsistent code review practices and no formal design review process.

T — Task

Improve code quality across the team without slowing down delivery velocity — and do it in a way that would outlast my own involvement.

A — Action

I introduced a structured code and design review process — not just reviewing PRs, but reviewing the design before implementation began. I created lightweight templates for design docs and ran a series of informal knowledge-sharing sessions to help the team internalise clean code principles and common failure patterns. I also advocated for TDD on new features and set up automated linting rules in the CI pipeline.

R — Result

Post-release bugs dropped by 30% within 6 months. The design review habit stuck — it became standard practice on the team. In the final 12 months I was at TRIAS, we had zero critical post-deployment regressions.

Story 6 — Onboarding & Mentoring

Use for: Leadership · Team development · Knowledge transfer · Multiplier effect

S — Situation

TRIAS was growing and bringing on new engineers — but the codebase was complex, domain-specific (healthcare), and had limited documentation. New engineers were taking a long time to become productive, which was slowing the whole team down.

T — Task

Reduce ramp-up time for new engineers while maintaining my own delivery commitments — and do it in a scalable way rather than just answering questions one-to-one.

A — Action

I created structured onboarding documentation covering the most complex parts of the codebase — architecture decisions, data model, key workflows, and common debugging patterns. I then ran a series of technical presentations and pair-programming sessions for the 4-5 new engineers joining the team. I also set up a lightweight mentoring structure where I'd review their first few PRs in depth with specific, constructive feedback rather than just approvals.

R — Result

New engineer ramp-up time dropped by 30%. The documentation became the team's standard reference for new joiners. Several of the engineers I mentored went on to own features independently within their first month — significantly ahead of the previous average.

Section 6: Interview Cheat Sheets

6.1 Your 90-Second Opening — Practise This Word for Word

Your 90-Second Opening Script

"I'm a Full Stack Software Engineer with just over three years of production experience, primarily at TRIAS — a healthcare SaaS startup in Bengaluru where I built cloud-native services on AWS, data pipelines using PySpark and Elasticsearch, and integrated Generative AI into live workflows.

The work I'm most proud of is the end-to-end ownership side — I shipped 15-plus features from architecture through to production across two international deployments, and built the CI/CD infrastructure that got us to zero downtime releases.

I then did an MSc in Data Analytics at NCI Dublin, which deepened my understanding of the data side — pipelines, analytical workloads, and how to build systems that turn raw events into decisions. To show that concretely, I built a full PySpark ETL pipeline on US cost-of-living data — ingest, transform, load to PostgreSQL, a statistical A/B experimentation engine, and a FastAPI serving layer. That's the kind of system the BE&O; team builds at scale.

Mastercard's BE&O; team caught my attention specifically because the work is exactly at the intersection of those two things — building scalable experimentation infrastructure that helps business users make data-driven decisions. That's the problem I want to be working on."

6.2 Technical SME Round

Lead with data pipeline experience — first 5 minutes

"I've built batch data pipelines processing 100k+ daily records at TRIAS using PySpark and Elasticsearch. Most recently I built a full ETL pipeline with PySpark — ingest to Parquet, broadcast join transforms, Spark SQL aggregation, load to PostgreSQL, and a statistical experimentation layer."

When they ask about Databricks specifically:

"Strong PySpark foundation — broadcast joins, partitioning, Catalyst optimiser, AQE. My code runs unchanged on Databricks. For Delta Lake: I'd swap Parquet writes for Delta writes to get ACID guarantees, time-travel, and streaming+batch on the same table. Actively deepening platform knowledge on Community Edition — Unity Catalog, Auto Loader, Workflows."

When they ask about experimentation / A/B testing:

"I built a pairwise experimentation engine in this project that uses Welch's t-test across all US Census regions. Each result has p-value, Cohen's d effect size, 95% CI, and lift %. I chose Welch's over Student's because regions have unequal sample sizes and variance. Cohen's d matters because p-value alone is misleading with large samples — you can get p=0.0001 with d=0.05 and the difference is practically meaningless."

System design: use the 7-step framework

1. Clarify requirements (functional + non-functional, scale)

2. Capacity estimation (data volume, QPS, storage)

3. API design (endpoints, request/response schemas)

4. Data model (tables, relationships, indexes)

5. High-level design (components + data flow diagram)

6. Deep dive on the hardest component

7. Trade-offs and alternatives considered

6.3 Bar Raiser Round — Go Two Levels Deep

The Bar Raiser will follow up on every claim. Prepare to go deeper on these:

If they ask about...	Surface answer (Level 1)	Deep answer (Level 2)
Idempotency	mode='overwrite' / if_exists='replace'	Partition-level overwrite for targeted reruns; Delta MERGE for row-level upserts
Spark performance	Broadcast joins, partitioning	AQE at runtime; skew handling with salting; push down filters before joins; cach
p-value	Probability of seeing this result by chance (<0.05 = significant)	Type I = false positive; Type II = false negative; Cohen's d for
Fault tolerance	try/except in ab_engine, DLQ pattern	Exactly-once vs at-least-once delivery; Spark checkpoint directories; Delta Lake
Java	Familiar — strong Python/Node.js background	Reviewed Java code at TRIAS for integration points. Fast learner track record:

6.4 Hiring Manager Behavioural Round

Why Mastercard? Why BE&O; specifically?

"Mastercard operates at a scale where every percentage point improvement in an experiment outcome translates to hundreds of millions of transactions. The BE&O; team is building the infrastructure that makes those decisions rigorous — not gut feel, but statistically validated. I built a version of that infrastructure in this project, and I want to do it at a scale and impact level that isn't possible anywhere else."

Why now — what makes you the right person?

"Three things: production data pipeline experience (PySpark, Elasticsearch, 100k+ daily records), I've shipped a complete experimentation platform in this project, and I'm available immediately — no notice period, no visa sponsorship cost."

Referral context — Rohan Anand (mention naturally if it comes up):

"Rohan referred me after we discussed the BE&O; team's work — specifically the experimentation platform they're building. It aligned directly with what I've been building independently, so the fit felt genuine rather than speculative."

6.5 Exact Sentences for Framing the Three Gaps

Gap 1: Databricks platform depth

SAY: "My PySpark foundation is solid — same API, same code. What I'm adding is the platform layer: Delta Lake for ACID guarantees and time-travel, Unity Catalog for governance, Workflows for orchestration. I'm working through this on Community Edition now and would expect to be productive on the platform within the first two weeks."

Gap 2: A/B testing / experimentation (actually a strength — frame it)

SAY: "I built a production-pattern experimentation engine in this project — Welch's t-test, Cohen's d, confidence intervals, lift percentage, winner determination. At TRIAS I ran experiments on the GenAI integration — before/after measurement across production traffic. I'm comfortable with the statistical core; what I'd be learning at Mastercard is the scale and the edge cases that come with billions of data points."

Gap 3: Java

SAY: "My primary languages are Python and Node/TypeScript. I've reviewed Java code for integration points at TRIAS. My track record with new languages is strong — as an intern I was productive in an unfamiliar codebase within 4 weeks with zero regressions. I'd prioritise Java syntax and idioms in the first sprint."

Section 7: Quick Reference Glossary

Plain-English definitions — 50 words max each. Scan this the morning of the interview.

Term	Plain-English Definition
Apache Spark	Distributed computing engine. Splits data across many machines and processes in parallel. Driver plans th
DAG	Directed Acyclic Graph. Spark's execution plan: a map of all transformations to run, in order, with no cycle
Parquet	Columnar file format. Stores data column-by-column. Faster for analytical queries (read only needed colur
Broadcast Join	Send a small table to ALL worker nodes so each can join locally. Eliminates network shuffle. Use when on
Predicate Pushdown	When Spark reads partitioned Parquet with a WHERE filter, it skips partitions that don't match — never rea
ETL	Extract-Transform-Load. Read raw data (Extract), clean and enrich it (Transform), write to target system (I
Medallion Architecture	Bronze (raw) → Silver (cleaned) → Gold (aggregated). Three-layer data lake pattern from Databricks. Eac
Idempotency	Running a pipeline twice gives the same result as running it once. No duplicate rows. Achieved with mode
A/B Test	Controlled experiment comparing a Control group (baseline) vs Treatment group (change). Statistical test
p-value	Probability of seeing this result if there were truly no difference (null hypothesis). $p < 0.05$: difference is sta
Alpha (α)	Significance threshold. If $p < \alpha$, reject the null hypothesis. Industry standard $\alpha = 0.05$ (5% false positive ra
Type I Error	False positive. Conclude there's a difference when there isn't. Rate = alpha (0.05).
Type II Error	False negative. Miss a real difference. Rate = beta. Statistical power = 1 - beta.
Welch's t-test	Two-sample t-test that does NOT assume equal variances. Use when groups have different sizes or sprea
Cohen's d	Effect size for a t-test: (mean_difference) / pooled_std. Small < 0.2, Medium < 0.5, Large ≥ 0.8. Tells you H
Confidence Interval	Range where the true value probably lies. 95% CI means: if you ran this experiment 100 times, 95 of thos
Lift %	(treatment_mean - control_mean) / control_mean × 100. Percentage change from baseline. Business-frien
Delta Lake	Parquet + a transaction log. Adds ACID transactions, time-travel (versioning), schema enforcement, and M
Kafka	Distributed message broker. Producers write events to topics; consumers read them. Durable, replayable,
Dead Letter Queue	Holding area for records that failed processing. Instead of crashing the pipeline, bad records are routed he
FastAPI	Modern Python web framework. Type hints + Pydantic = auto-validation + auto Swagger docs. Faster than
Pydantic	Python library for data validation using type hints. Define a model; Pydantic validates input automatically. U
SQLAlchemy	Python SQL toolkit and ORM. Provides DB-agnostic connection, parameterised queries (SQL injection pre
Dagster	Data orchestration framework. Wraps pipeline steps as typed ops, manages run history, lineage, retries. L
Structured Streaming	Spark's streaming API. Reads from Kafka/files as a continuous stream, applies the same DataFrame trans
AQE (Adaptive Query Execution)	Spark feature that re-optimises the execution plan at runtime using actual data statistics. Handles skew, a

Good luck, Idhaya. You've built the project. You know the stats. You have the stories. Go get it. ■